

```
In [12]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Set aesthetic parameters for seaborn
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams.update({'figure.figsize': (10, 6), 'figure.dpi': 100})

# Load the dataset
dataset_path = 'iris.csv'
data = pd.read_csv(dataset_path)

# Display the first few rows
display(data.head())
```

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [13]: # Dataset Info and Description
print("--- Dataset Info ---")
display(data.info())
print("\n--- Dataset Description ---")
display(data.describe())
print("\n--- Missing Values ---")
display(data.isnull().sum())
```

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   SepalLength     150 non-null   float64
1   SepalWidth      150 non-null   float64
2   PetalLength     150 non-null   float64
3   PetalWidth      150 non-null   float64
4   Name            150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
--- Dataset Description ---
```

	SepalLength	SepalWidth	PetalLength	PetalWidth
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

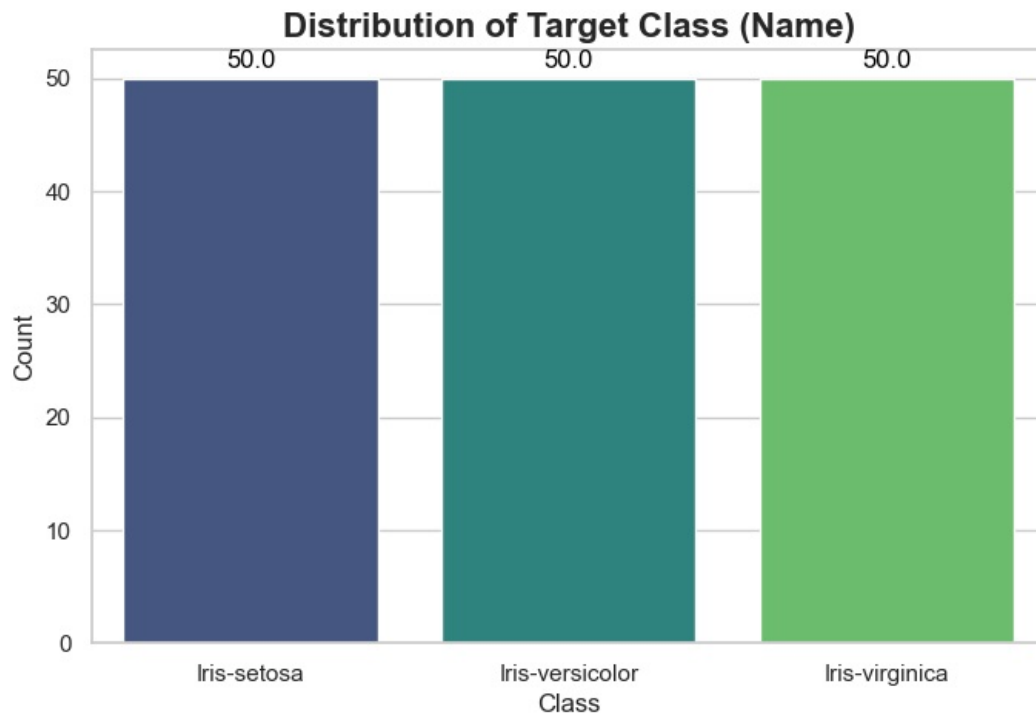
```
--- Missing Values ---
SepalLength    0
SepalWidth     0
PetalLength    0
PetalWidth     0
Name           0
dtype: int64
```

```
In [14]: # Distribution of the target class
plt.figure(figsize=(8, 5))
ax = sns.countplot(data=data, x='Name', palette='viridis')
plt.title('Distribution of Target Class (Name)', fontsize=16, fontweight='bold')
```

```

plt.xlabel('Class', fontsize=12)
plt.ylabel('Count', fontsize=12)
for p in ax.patches:
    ax.annotate(f'{p.get_height()}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='baseline', fontsize=12, color='black', xytext=(0, 5),
                textcoords='offset points')
plt.show()

```

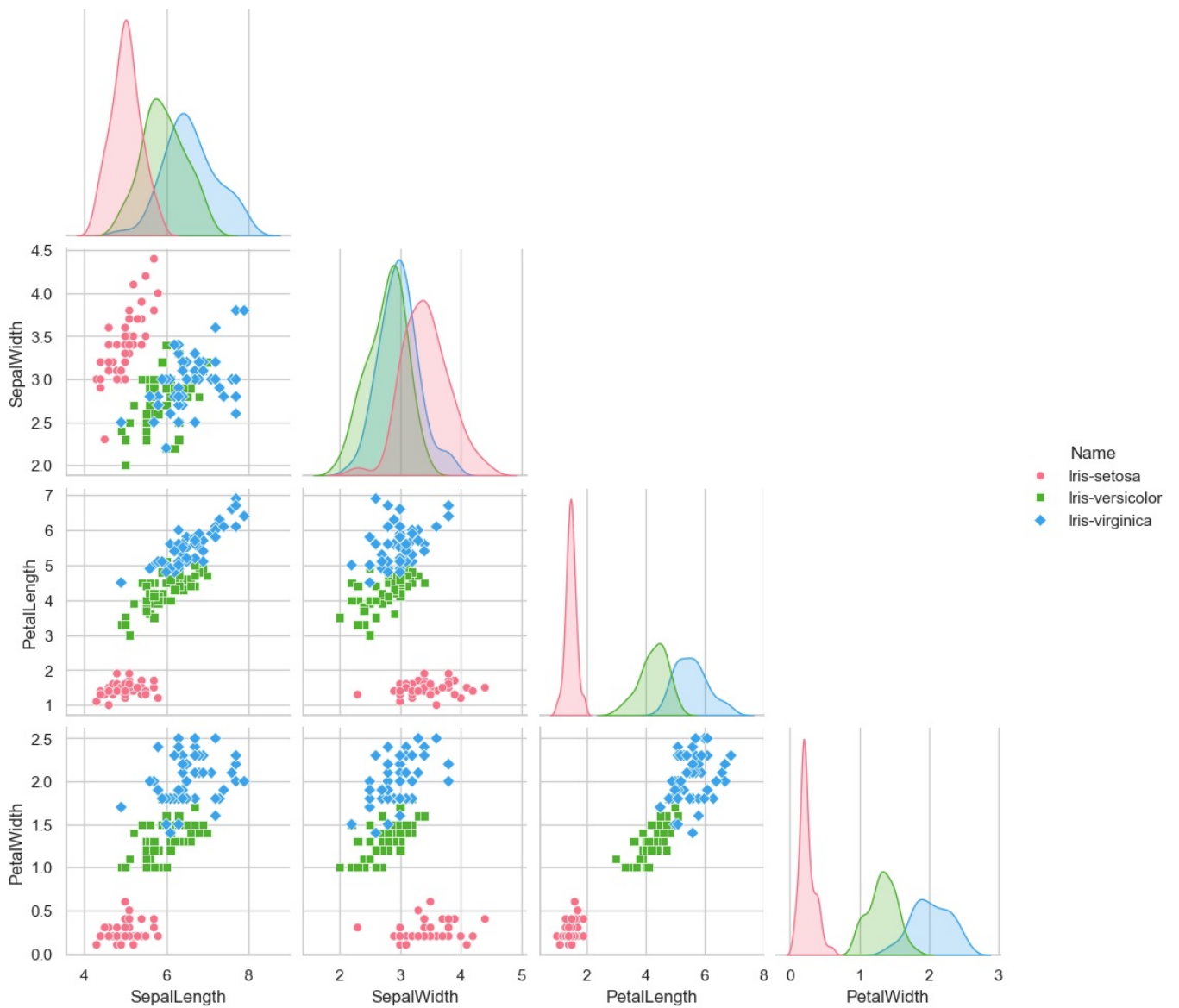


```

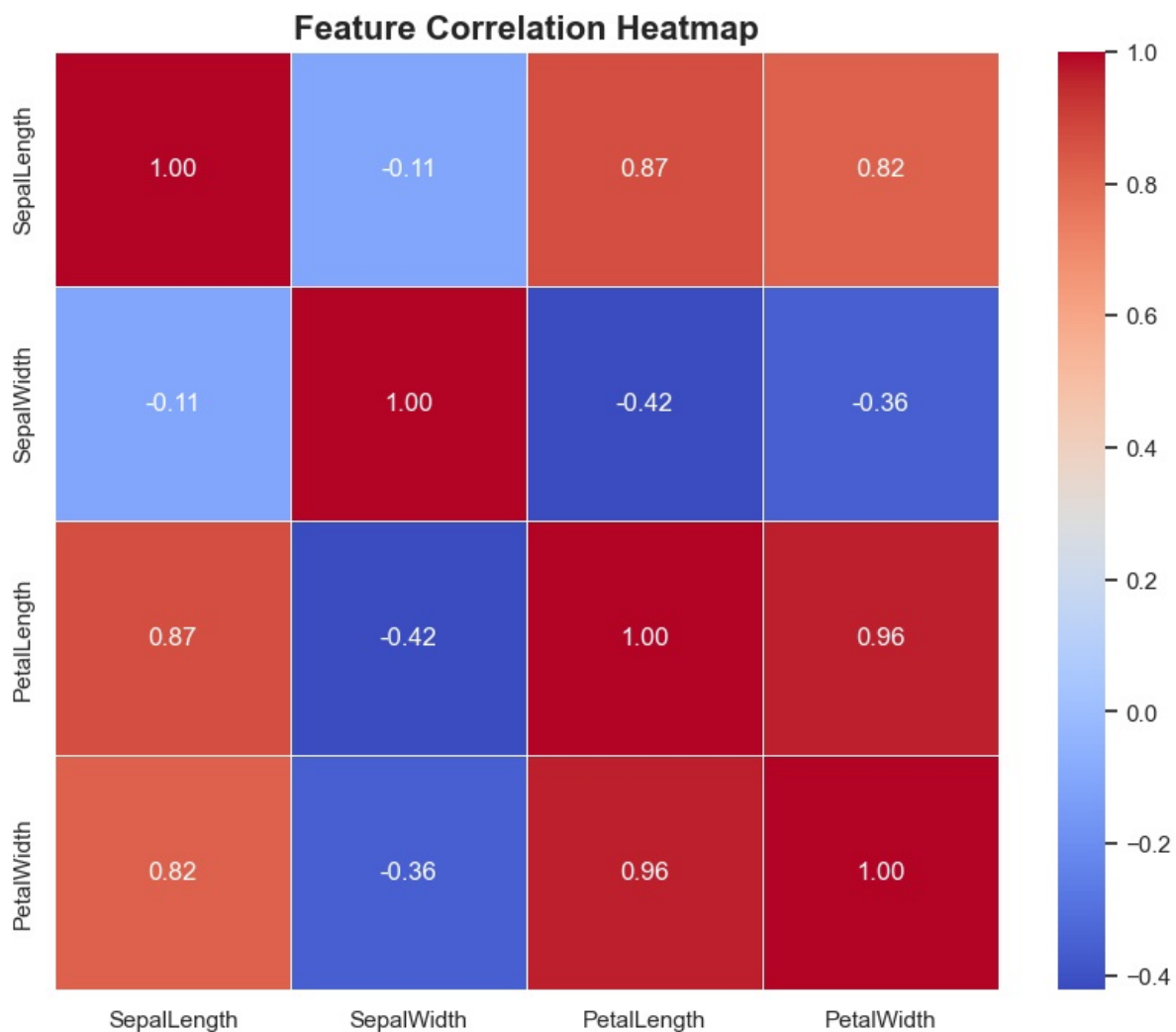
In [15]: # Pairplot to visualize feature relationships
sns.pairplot(data, hue='Name', palette='husl', markers=['o', 's', 'D'], corner=True)
plt.suptitle('Pairplot of Features by Class', y=1.02, fontsize=16, fontweight='bold')
plt.show()

```

Pairplot of Features by Class



```
In [16]: # Correlation Heatmap (excluding non-numeric columns)
plt.figure(figsize=(10, 8))
numeric_data = data.select_dtypes(include=[np.number])
correlation_matrix = numeric_data.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5, annot_kws={"size": 12})
plt.title('Feature Correlation Heatmap', fontsize=16, fontweight='bold')
plt.show()
```



```
In [17]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
```

```
# Predictions
y_pred = logr.predict(X_test)

# Calculate Metrics (using 'weighted' average for multiclass)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print("--- Classification Report ---")
print(classification_report(y_test, y_pred))

# Visualize Metrics
metrics_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
metrics_values = [accuracy, precision, recall, f1]

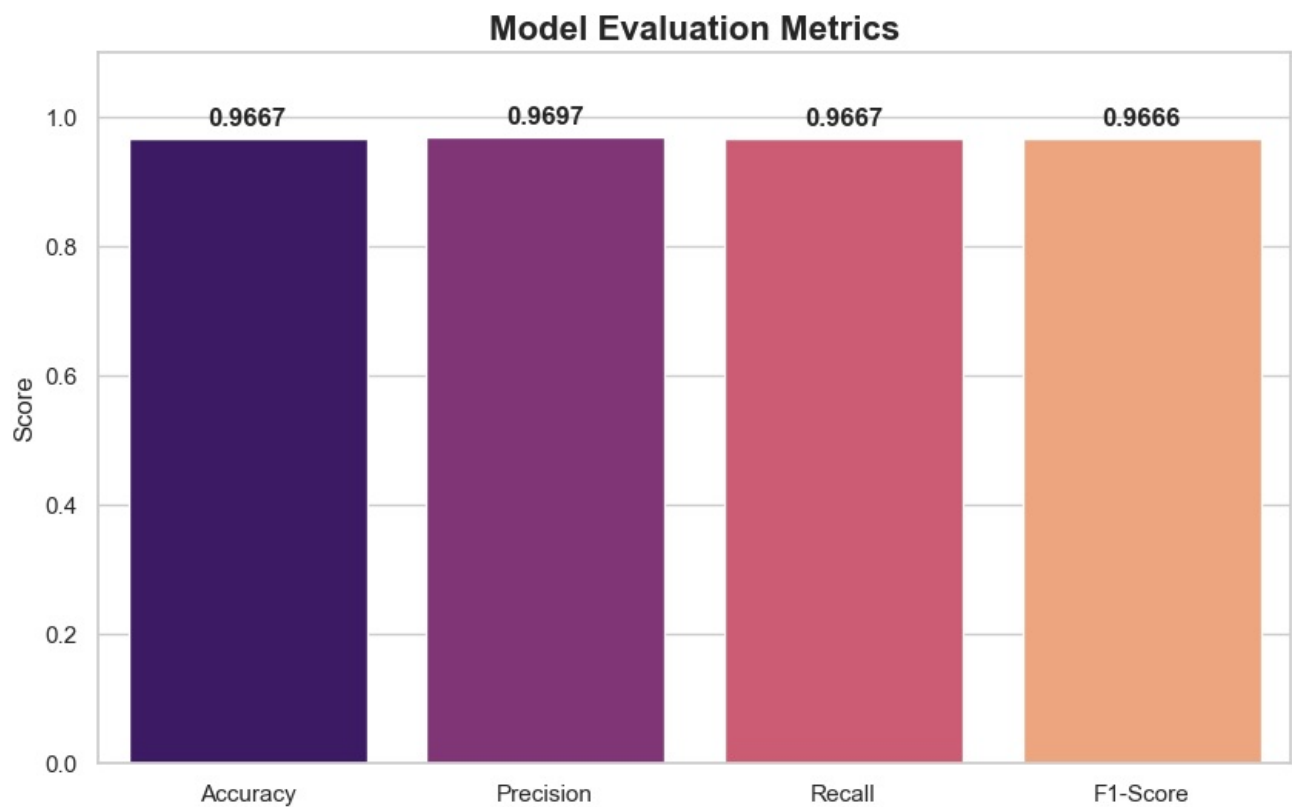
plt.figure(figsize=(10, 6))
ax = sns.barplot(x=metrics_names, y=metrics_values, palette='magma')
plt.title('Model Evaluation Metrics', fontsize=16, fontweight='bold')
plt.ylim(0, 1.1)
plt.ylabel('Score', fontsize=12)

# Add value labels
for i, v in enumerate(metrics_values):
    ax.text(i, v + 0.02, f"{v:.4f}", ha='center', fontweight='bold', fontsize=12)

plt.show()
```

```
--- Classification Report ---
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	1.00	0.90	0.95	10
Iris-virginica	0.91	1.00	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30



```
In [18]: from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
from itertools import cycle

# Binarize the output for multiclass ROC
y_test_bin = label_binarize(y_test, classes=logr.classes_)
n_classes = y_test_bin.shape[1]

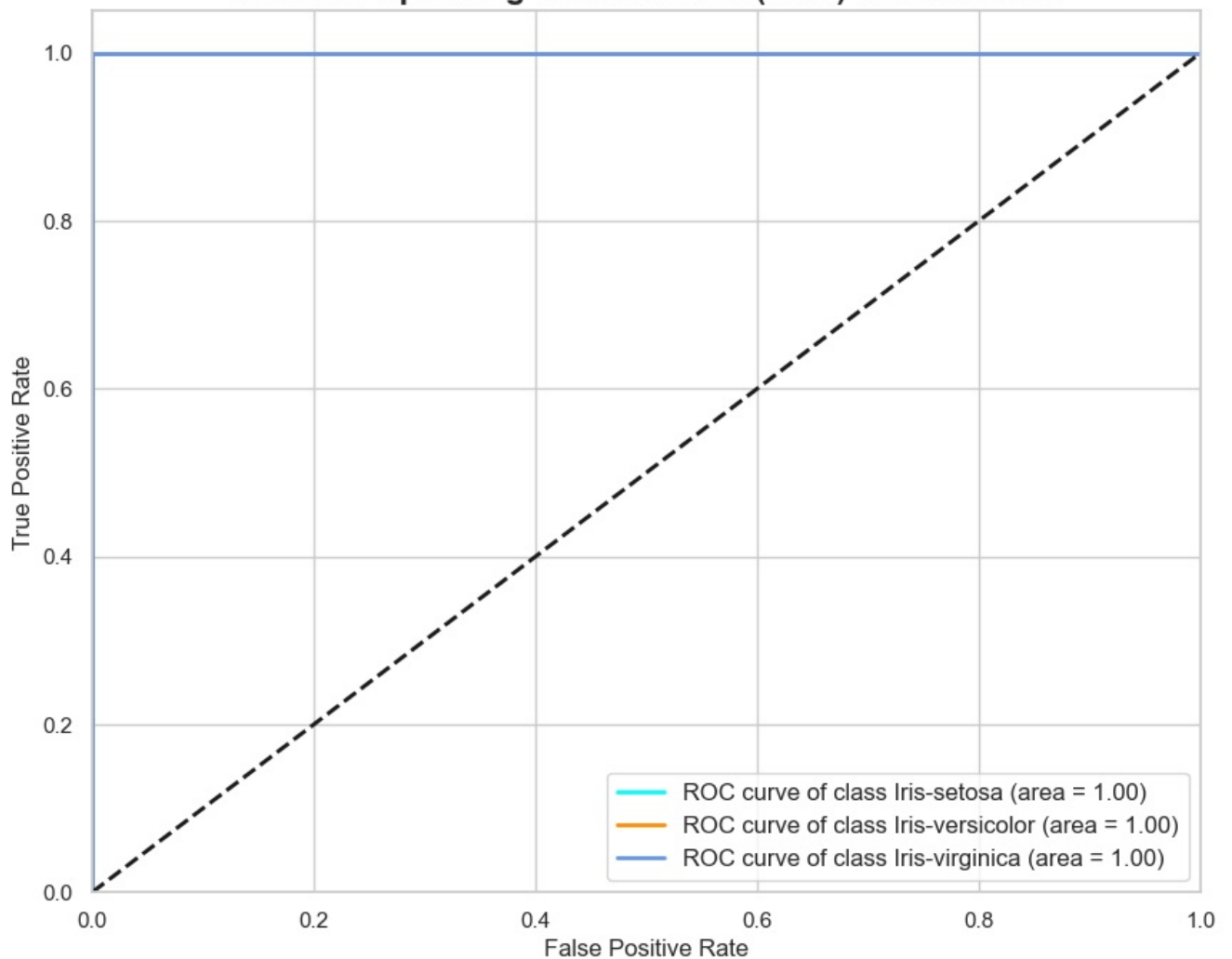
# Predict probabilities
y_score = logr.predict_proba(X_test)

# Compute ROC curve and ROC area for each class
fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot all ROC curves
plt.figure(figsize=(10, 8))
colors = cycle(['aqua', 'darkorange', 'cornflowerblue'])
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label=f'ROC curve of class {logr.classes_[i]} (area = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('Receiver Operating Characteristic (ROC) for Multiclass', fontsize=16, fontweight='bold')
plt.legend(loc="lower right", fontsize=12)
plt.show()
```

Receiver Operating Characteristic (ROC) for Multiclass



```
In [19]: # Real-World Prediction
# Taking a sample 2D Python list of new data
sample_data = [[5.0, 3.6, 1.4, 0.2]]

# Convert to Pandas DataFrame using training column names to avoid warnings
sample_df = pd.DataFrame(sample_data, columns=X.columns)

# Output the predicted class and confidence probabilities
predicted_class = logr.predict(sample_df)[0]
confidence_probabilities = logr.predict_proba(sample_df)[0]

print("--- Real-World Prediction ---")
print(f"Sample Input Data:\n{sample_df}\n")
print(f"Predicted Class: {predicted_class}")
print(f"Confidence Probabilities:")
for cls, prob in zip(logr.classes_, confidence_probabilities):
    print(f" - Class {cls}: {prob*100:.2f}%")
```

--- Real-World Prediction ---

Sample Input Data:

	SepalLength	SepalWidth	PetalLength	PetalWidth
0	5.0	3.6	1.4	0.2

Predicted Class: Iris-setosa

Confidence Probabilities:

- Class Iris-setosa: 98.27%
- Class Iris-versicolor: 1.73%
- Class Iris-virginica: 0.00%